Universidad Distrital Francisco José de Caldas
School of Engineering

Real-Time Air Quality Monitoring and Recommendation System: A Data-Driven Software Approach

Johan Esteban Castaño Martínez

Stivel Pinilla Puerta

Professor: Carlos Andrés Sierra

Report submitted in partial fulfilment of the requirements of
the course *SDatabases Systems* at the
Universidad Distrital Francisco José de Caldas
as part of the undergraduate program in
Computer Engineering

July 10, 2025

Declaration

We, Johan Esteban Castaño Martínez and Stivel Pinilla Puerta,
of the Department of Computer Engineering, Universidad Distrital Francisco José de Caldas, confirm that this is our own work and that all figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

We give consent for our work to be made available more widely to members of the university and the public with interest in teaching, learning, and research.

Johan Esteban Castaño Martínez
Stivel Pinilla Puerta
July 10, 2025

Abstract

This project presents the design and implementation of an air quality data platform developed as part of the Database II course at Universidad Distrital. The system ingests real-time environmental data from multiple external sources every 10 minutes, storing historical records using PostgreSQL extensions. Key architectural strategies include temporal and geographic partitioning, read replicas for performance isolation, and materialized views to support real-time dashboards and historical reporting.

To ensure scalability, availability and responsiveness under concurrent access, the platform adopts asynchronous ingest pipelines and proposes future integration of message queues. Extensive concurrency analysis highlights potential risks such as missed updates, dirty reads, and deadlocks, with proposed control mechanisms including optimistic locking, row-level locking, and concurrent updating of materialized views.

The system supports multiple user roles-citizens, researchers, and public entities-and offers tailored visualizations and advanced query capabilities. Through eight SQL query templates, the platform enables rapid access to current AQI data, longitudinal contaminant trends, and raw intake payload scrubbing. This work demonstrates

Keywords: Air quality monitoring, real-time data ingestion, distributed databases, SQL query optimization, concurrency control.

Contents

List of Figures

List of Tables

List of Abbreviations

AQI	Air Quality Index
API	Application Programming Interface
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
GIS	Geographic Information System
UI	User Interface
UX	User Experience
PK	Primary Key
FK	Foreign Key
SLD	Styled Layer Descriptor
ETL	Extract, Transform, Load
CRUD	Create, Read, Update, Delete
KPI	Key Performance Indicator
OLAP	Online Analytical Processing
DBMS	Database Management System
IoT	Internet of Things
PMBOK	Project Management Body of Knowledge
UML	Unified Modeling Language

Chapter 1

Introduction

1.1 System Description

The *Air Quality Analysis Platform* is a distributed and modular system for monitoring, analyzing, and visualizing real-time and historical air quality data. It collects environmental data every 10 minutes from various APIs, such as AQICN, Google, IQAir, and local stations, and stores it in a PostgreSQL database extended with TimescaleDB for efficient time-series handling. The system is designed to support different user profiles, including citizens, researchers, and public agencies, offering them dashboards, reports, and personalized alerts.

1.2 System Components

Component	Main Function
Data Ingestor	Fetches and processes air quality data from multiple APIs every 10 minutes, applying validation and normalization.
Database Engine	Stores millions of records using time and geographic partitioning for optimal access and scalability.
SQL Queries	Enables fast access to current and historical data using materialized views and optimized indexes.
REST/GraphQL API	Provides programmatic access for users and services to retrieve dashboard data or reports.
Web/Mobile UI	Visualizes live AQI indicators, long-term trends, and configurable alerts per user profile.
Backup and Recovery	Uses MinIO to persist raw JSON payloads for traceability and data reprocessing if needed.

Table 1.1: Core System Components

1.3 Inputs, Processes, and Outputs

Inputs:

- Real-time air quality metrics (e.g., PM2.5, PM10, NO2) from third-party APIs.
- User-generated data: alert preferences, locations of interest, and configurations.

Processes:

- Data ingestion, deduplication, partition routing.
- Aggregation and materialization of historical and real-time views.
- Concurrent access management and role-based data segmentation.

Outputs:

- Interactive dashboards with updated AQI per city.
- Personalized reports and environmental alerts.
- Raw data inspection and downloadable series for researchers.

1.4 Stakeholders and System Actors

- **Citizens**: Access air quality dashboards, receive alerts, and make informed health decisions.
- **Researchers**: Download historical data and conduct environmental analysis.
- **Public agencies**: Generate reports and support air quality policy decisions.
- **Administrators**: Monitor ingestion pipelines and debug JSON payloads.

1.5 System Requirements

Functional Requirements:

- Display real-time AQI per city.
- Query historical pollutant trends.
- Enable alert configuration and dashboard customization.

Non-functional Requirements:

- High availability and fault tolerance.
- Sub-second response time for frequent queries.
- Secure, role-based access control.
- Scalable ingestion and storage architecture.

1.6 Concurrency and Interaction Model

The system faces challenges in managing concurrent operations such as real-time ingestion, dashboard access, and alert personalization. Specific concurrency risks include:

- *Lost updates*: Conflicting alert updates from different devices.
- *Dirty reads*: Accessing materialized views mid-refresh.
- *Deadlocks*: Simultaneous updates across metadata and alert tables.

To mitigate these, the platform uses:

- `REFRESH MATERIALIZED VIEW CONCURRENTLY`
- Optimistic locking with `last_updated` fields.
- Row-level locking with `SELECT ... FOR UPDATE`
- Unique constraints and transactions on ingestion.

1.7 Limitations and Future Directions

Current Limitations:

- Materialized view refresh may cause latency spikes.
- Partition management complexity increases with data volume.
- No message queue system implemented yet.

Future Enhancements:

- Introduce Kafka or RabbitMQ for asynchronous ingestion pipelines.
- Add caching layers (e.g., Redis) for real-time dashboards.
- Deploy regional read replicas for load balancing.

Chapter 2

Business and Technological Context

2.1 Business Model Overview

The Air Quality Analysis Platform operates under a data-driven business model aimed at providing environmental intelligence to a broad set of stakeholders. Its design integrates spatial databases, big data techniques, and business intelligence (BI) to deliver real-time air quality insights, reports, and recommendations.

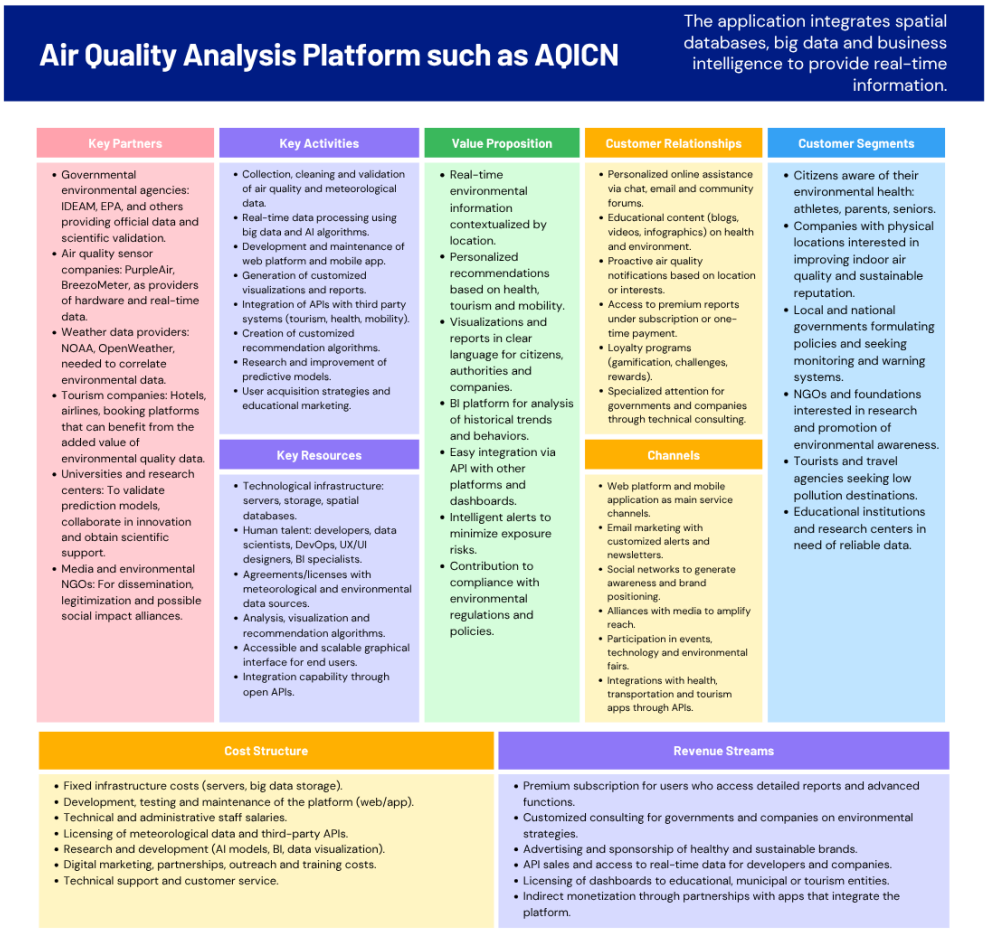

Figure 2.1: Business Model Canvas of the Air Quality Analysis Platform

2.1.1 Cost Structure and Revenue Streams

- **Cost Structure:**
  - Servers and storage for large-scale environmental data.
  - Platform development, maintenance, and DevOps operations.
  - Licenses and API access to external meteorological data sources.
  - Salaries for technical personnel (data engineers, developers, BI analysts).
  - R&D efforts in AI and environmental data science.
  - Marketing and user acquisition campaigns.
- **Revenue Streams:**
  - Paid access to historical data and customized analytical reports.
  - Advertising and sponsorship from health and environmental brands.
  - Consultancy and advisory services for governments and corporations.

2.1.2 Key Partners and Activities

- **Key Partners:**
  - Government agencies (e.g., IDEAM, EPA).
  - Air quality sensor manufacturers (PurpleAir, BreezoMeter).
  - Weather data providers (NOAA, OpenWeather).
  - Tourism industry stakeholders (hotels, airlines, travel platforms).
- **Key Activities:**
  - Environmental data ingestion, processing, and long-term storage.
  - Personalized recommendation engine based on air quality levels.
  - Real-time dashboards and geographic visualizations.
  - Report generation for institutions and citizens.
  - Integration with third-party systems (health, transport, weather).

2.1.3 Value Proposition and Customer Relationships

- **Value Proposition:**
  - Real-time air quality monitoring and alerts.
  - BI tools for trend analysis and urban planning.
  - Personalized recommendations for safer mobility and lifestyle.
- **Customer Relationships:**
  - Online support and feedback channels.
  - Platform-generated notifications and alert systems.
  - Community engagement through educational content.
  - Tailored reports for users, NGOs, and public sector entities.

2.1.4 Customer Segments and Channels

- **Customer Segments:**
  - Health-conscious citizens (e.g., families, runners, elderly).
  - Companies monitoring indoor/outdoor air conditions.
  - Local and national governments implementing air quality policies.
  - NGOs and researchers studying environmental health.
  - Tourists choosing destinations based on environmental factors.
- **Channels:**
  - Web platform and mobile app.
  - API access for integration with external systems.
  - Email campaigns and push notifications.
  - Social media and environmental outreach partnerships.

2.2 How the Platform Works

The platform follows a three-stage operational workflow:

1. **Data Collection:** From real-time sensors and APIs.
2. **Data Processing:** Includes validation, transformation, deduplication, and partitioned storage.
3. **Data Presentation:** Via dashboards, alerts, reports, and maps.

2.3 Use Cases and Target Users

- **Citizens:** Want to know whether it is safe to be outdoors.
- **Governments:** Issue alerts and formulate air quality policies.
- **Companies and NGOs:** Analyze environmental impact and plan mitigation strategies.

2.4 Technological Ecosystem

- Relational databases (PostgreSQL with TimescaleDB) and NoSQL for unstructured data.
- BI modules for analytical dashboards and trend visualization.
- RESTful and GraphQL APIs for multi-platform access.
- Architecture built for fault tolerance, high availability, and distributed deployment.

Chapter 3

Methodology

This chapter describes the methodological approach used in the development of the Air Quality Analysis Platform. It includes the specification of functional and non-functional requirements, the software engineering process followed, and the technologies applied to implement a scalable, distributed, and user-focused solution.

3.1 Requirements Specification

The requirements of the platform were identified based on user needs, project objectives, and technological constraints. These are divided into functional and non-functional requirements and are associated with specific user stories.

3.1.1 Functional Requirements

ID	Requirement	Associated User Stories
FR1	The system must collect real-time air quality data from multiple external sources (e.g., APIs, stations) and store it for further processing.	US1, US13, US14
FR2	The system must allow users to query and visualize historical air quality data filtered by date, location, and pollutant type.	US2, US7
FR3	The system must display air quality information in a uniform and clear manner, regardless of its origin.	US1, US3
FR4	The system must display real-time dashboards with key performance indicators (KPIs) on air quality.	US4
FR5	The system must generate customized reports with filters for date, location, and indicators, available for download or email.	US5
FR6	The system must present interactive graphs showing air quality evolution over time.	US6
FR7	The system must allow users to export historical data in standard formats like CSV and JSON.	US7
FR8	The system must provide personalized recommendations based on user location and air quality conditions.	US8
FR9	The system must send customizable air quality alerts when thresholds are exceeded.	US9

ID	Requirement	Associated User Stories
FR10	The system must suggest certified protective products during high pollution periods.	US10
FR11	The system must display maps highlighting areas with better air quality for navigation.	US11
FR12	The system must support geographic search for air quality data by country, city, or region.	US15
FR13	The system must be responsive and compatible with mobile, tablet, and desktop devices.	US16
FR14	The system must allow users to share air quality data and reports on social media with pre-generated links and previews.	US17

3.1.2 Non-Functional Requirements

ID	Requirement	Associated User Stories
NFR1	Queries on large datasets (1 million records) must execute in under 2 seconds 95% of the time.	US3, US12
NFR2	The system must support continuous streaming data ingestion 24/7 without manual intervention.	US1, US14
NFR3	Data storage must be distributed and optimized for big data processing.	US1, US3
NFR4	Customized reports must be generated in under 10 seconds.	US5
NFR5	The recommendation engine must update in real time as air quality data changes.	US8, US10
NFR6	Air quality data and visualizations must load in less than 2 seconds for 95% of user requests.	US12
NFR7	The system architecture must include fault tolerance and geographic redundancy.	US14
NFR8	The system must scale horizontally to support growth in data volume and users.	US13, US14
NFR9	The system UI must function correctly on all major browsers and operating systems without errors.	US16
NFR10	Data consistency and user personalization must be preserved across all user devices.	US16

Note: Some of the requirements were refined based on feasibility, estimated performance expectations, and end-user priorities.

3.2 Development Approach

The project followed an iterative and incremental development process, allowing for early testing of features, continuous feedback incorporation, and modular deployment.

3.2.1 Tools and Technologies Used

- **Database Layer:** PostgreSQL (with PostGIS) for spatial queries and MongoDB for real-time NoSQL data.
- **Backend:** Developed in Go (Golang), enabling high performance and concurrency for data ingestion.
- **Frontend:** Web interface built with React and OpenLayers for geospatial visualizations.
- **Data Sources:** Integration with AQICN, OpenWeather, and Google Maps APIs.
- **BI Layer:** Dashboards and visualizations generated using custom BI components and libraries.

3.2.2 System Architecture

The system is designed as a distributed architecture with microservices, supporting:

- Real-time data streaming and storage.
- Load balancing and failover mechanisms.
- API access for third-party systems.
- Modular scalability across regions.

3.3 Ethical Considerations

- **Privacy:** All user data is anonymized and encrypted to ensure confidentiality.
- **Consent:** Explicit consent is required for personalized alerts and location-based services.
- **Compliance:** The system complies with data protection regulations and environmental data usage policies.
- **Transparency:** Data sources are clearly indicated, and limitations are disclosed.
- **Social Impact:** The platform promotes public health awareness and supports environmental policy-making.

3.4 User Stories

This section presents the user stories identified for the development of the Air Quality Analysis Platform. Each user story includes a role, a specific need, and clear acceptance criteria that guide implementation and testing. These stories reflect the expectations of different types of users including citizens, researchers, technical administrators, and public policy makers.

User Story ID	Role and Need	Acceptance Criteria
US1	As a Technical Administrator, I want to collect real-time data from multiple APIs so that I can provide accurate and up-to-date information on air quality.	The system receives updates at least every 10 minutes from configured sources. Processes at least 1000 data points per minute without loss. Logs successful and failed ingestions with error codes.
US2	As a Researcher/Analyst, I want to access historical data by date and location so that I can perform longitudinal analysis and scientific research.	User can filter by city, date, and pollutant type. Data export available in CSV and JSON. Historical records available for up to 3 years.
US3	As a Technical Administrator, I want to run queries over large volumes of data quickly so that I can avoid delays when searching for information.	Queries over 1 million records return in under 3 seconds. Indexes and partitions are used for performance.
US4	As a Public Policy Manager, I want to access dashboards with real-time analysis so that I can issue alerts or recommendations to the public.	Dashboards include real-time maps, graphs, and alerts. Automatic refresh without manual reload. Critical thresholds can be configured for alerts.
US5	As a Public Policy Manager, I want to generate customized reports on pollution trends so that I can design evidence-based public policies.	User can choose indicators, date ranges, and export formats. Reports are downloadable in PDF or sent via email. Graphs are auto-generated from selected data.
US6	As a Citizen, I want to view interactive graphs about air quality evolution so that I can understand changes and make informed decisions.	Filters for location, date, and pollutant available. Charts update dynamically with parameter changes. Visualizations load in under 2 seconds.
US7	As a Researcher/Analyst, I want to export historical data in standard formats so that I can analyze it with my own statistical tools.	Interface allows selection of location, date, and format. Files download successfully without errors. Download limits prevent system overload.
US8	As a Citizen, I want to receive personalized recommendations based on air quality so that I can know if it is safe to do outdoor activities.	Location and user preferences are considered. Alert color coding (green, yellow, red) is used. Suggested actions are clearly displayed.
US9	As a Citizen, I want to receive early warnings when air quality is harmful so that I can take precautions in time.	User can set alerts by pollutant and critical threshold. Notifications sent via email. Alert triggers when AQI surpasses configured limits.
US10	As a Citizen, I want to receive suggestions for protective products so that I can protect my health during high pollution levels.	Suggestions appear only during high pollution periods. Products are certified and include links. User can disable this feature in preferences.
US11	As a Citizen, I want to see areas with better air quality so that I can avoid the most polluted zones when moving.	System displays a map with AQI levels per area. Users can compare different city places.

User Story ID	Role and Need	Acceptance Criteria
US12	As a Citizen, I want to load air quality data quickly so that I can access information without delay.	Air quality data loads in under 2 seconds normally. System handles 100 concurrent users without slowdown. Cache is used for frequently accessed queries.
US13	As a Technical Administrator, I want to handle traffic peaks without failure so that I can maintain user experience during high demand.	Stress test simulates 1000 concurrent users. System remains responsive under load. Latency remains below 5 seconds per request.
US14	As a Technical Administrator, I want to keep the platform available 24/7 so that I can ensure constant access to information.	Platform includes geographic redundancy. Uptime monitoring with automated alerts is enabled. Monthly availability is 99.9%.
US15	As a Citizen, I want to check air quality in different regions so that I can plan trips and activities.	Users can switch between countries/regions easily. Information is shown in local language where possible.
US16	As a Citizen, I want to access the platform from various devices so that I can always access data regardless of the device.	Responsive design across mobile, tablet, and desktop. Core functions work on all devices.
US17	As a Citizen, I want to share air quality data on social media so that I can inform and raise awareness among others.	Sharing available on X, Facebook, Instagram, WhatsApp. Preview includes summary and image. Links and QR codes are auto-generated for sharing.

Table 3.3: User Stories

3.5 ER Diagram

This section presents the Entity-Relationship (ER) diagram of the system, which models the key components and their relationships. It provides a high-level overview of the data structure and the organization of the database entities that support the functionalities described in the user stories and requirements.

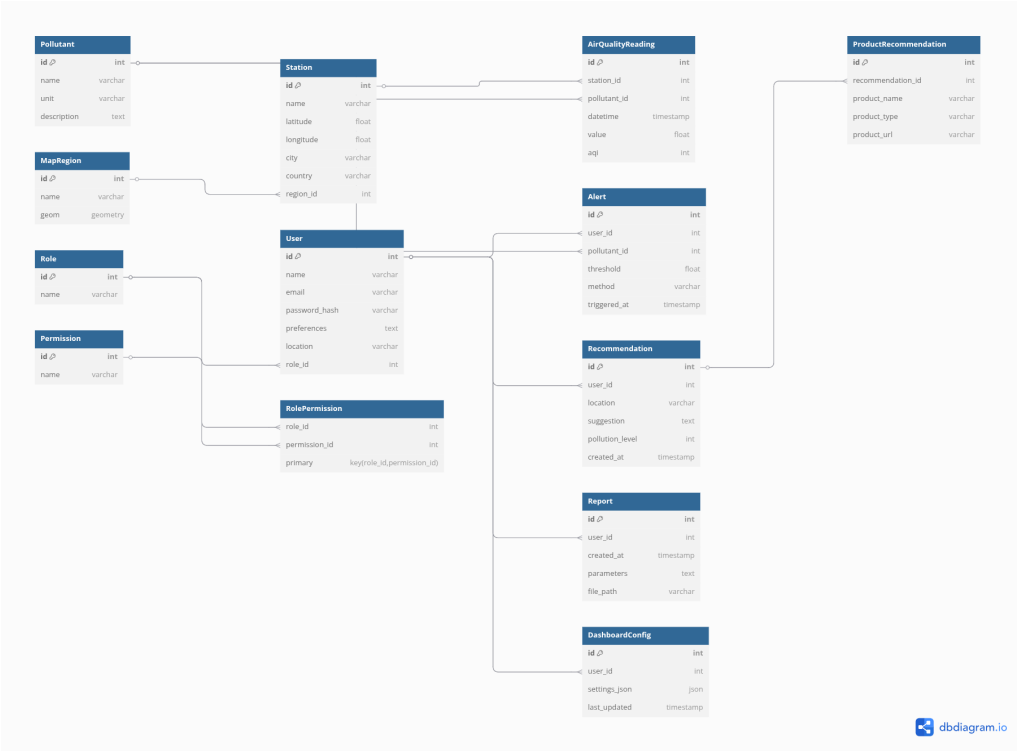

Figure 3.1: Entity Relationship Diagram

Component	Entity Name	Description	Attributes
Geospatial Layer	Station	Monitoring station metadata and location.	id (PK), name, latitude, longitude, city, country
Geospatial Layer	AirQuality Reading	Air quality measurements from stations.	id (PK), station_id (FK), datetime, pollutant_type, value, aqi
Geospatial Layer	Pollutant	Catalog of pollutants and their units.	id (PK), name, unit, description
Customer Layer	User	Platform user and profile settings.	id (PK), name, email, password_hash, preferences, location
Recommendation Engine	Alert	Alert configurations and triggered events.	id (PK), user_id (FK), pollutant_type, threshold, method, triggered_at

Component	Entity Name	Description	Attributes
Recommendation Engine	Recommendation	User-specific recommendations based on AQI.	id (PK), user_id (FK), location, suggestion, pollution_level, created_at
Recommendation Engine	Report	Generated analytical reports.	id (PK), user_id (FK), created_at, parameters, file_path
Geospatial Layer	MapRegion	Regions used for geospatial filtering and navigation.	id (PK), name, geom (geometry)
Customer Layer	Dashboard Config	User preferences for dashboard visualizations.	id (PK), user_id (FK), settings_json, last_updated
Recommendation Engine	Product Recommendation	Suggested certified products during high pollution levels.	id (PK), recommendation_id (FK), product_name, product_type, product_url
Customer Layer	Role	Defines user roles in the platform (e.g., Admin, Citizen, Analyst).	id (PK), name (unique)
Customer Layer	Permission	Specifies distinct permissions that can be granted to roles.	id (PK), name (unique)
Customer Layer	Role Permission	Associates roles with permissions in a many-to-many relationship.	role_id (FK), permission_id (FK), PRIMARY KEY (role_id, permission_id)

Table 3.4: Entities in the ER Diagram

3.6 Data System Architecture – Explanation

This section describes the architecture of the data system that supports the platform's core functionality, from ingestion to user access. The design prioritizes real-time processing, scalability, fault tolerance, and performance optimization.

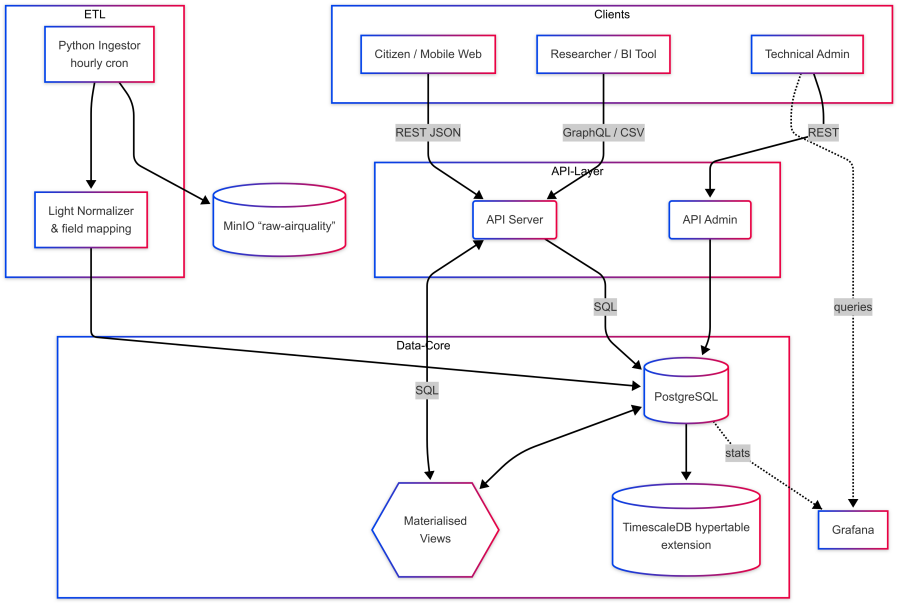

Figure 3.2: High-level architecture diagram

1. Ingestion and Raw Storage

A lightweight Python service (**Ingestor**) polls air quality endpoints at regular intervals. Each JSON payload is first persisted—unchanged—in a MinIO bucket (`raw-airquality`) to ensure auditability and allow for future replay in case of schema changes or ingestion failures.

2. Normalization and Relational Store

A normalization layer maps heterogeneous field names, units, and AQI standards into a unified format. This harmonized data is inserted into a partitioned PostgreSQL database using TimescaleDB extensions. The monthly time partitions reduce index size, improve cache hits, and allow for rapid time-series filtering.

3. Query Acceleration

To accelerate analytical queries, the system uses **materialized views** that store pre-aggregated metrics such as daily AQI averages or pollutant frequency histograms. These views are refreshed concurrently to avoid downtime or blocking operations, ensuring that dashboards and reports return data promptly.

4. API and User Access

A flexible API layer built on REST and GraphQL provides access to the normalized and aggregated data. Citizens, researchers, and third-party systems consume this data via web interfaces or external clients. Role-based access ensures administrative and analytical queries are routed through a dedicated secure API. Query latency, API errors, and ingestion failures are monitored continuously using Grafana and Prometheus dashboards.

3.7 Information Requirements

This section defines the information needs of different actors and technical components of the platform. These requirements are categorized as Organizational, Asset, Project, and Exchange Information Requirements, helping guide decisions on infrastructure, interoperability, and development priorities.

2.1 Organizational Information Requirements

OIR	Description
District-level air quality indicators	To meet sustainability, regulatory compliance, and policy formulation objectives.
Historical trends and longitudinal analysis	To evaluate the impact of environmental public policies over time.
System performance metrics (uptime, latency, scalability)	To ensure service continuity and operational quality.
Aggregate user behavior data	To optimize the digital experience, educational campaigns, and loyalty strategy.
Information on successful and unsuccessful integrations with third parties (API partners)	For technological or commercial improvement and expansion decisions.
Premium feature usage statistics	To evaluate revenue models and retention strategies.

2.2 Asset Information Requirements

AIR	Description
Server and database specifications (PostgreSQL, clusters)	To ensure processing and storage capacity.
System health status logs (processes, microservices, containers)	For predictive maintenance and stability monitoring.
Network configurations, load balancers, geographic redundancy	To ensure availability and fault tolerance.
Technical information from sensors and external data sources (frequency, format, authentication)	To maintain integrity and consistency of data ingestion.
Active and current licenses (third-party APIs, BI software like Metabase)	To ensure legal and technical software compliance.
Details of internal and external APIs, technical documentation, endpoint versioning	For maintenance and integration with new systems.

2.3 Project Information Requirements

PIR	Description
Deployment schedules for new modules (recommendations, BI, alerts)	For release planning and team coordination.
User story acceptance and validation criteria	For functional testing and compliance verification.
Backend technical specifications (APIs, pipelines)	For modular and scalable development.
Testing data, staging and validation environments	For automated testing and quality control before deployment.
Alerts and dashboards configuration documentation	To ensure that institutional customers receive the contracted features.
Evidence of compliance with legal and regulatory requirements (environmental, privacy)	To ensure regulatory adherence during development.

2.4 Exchange Information Requirements

EIR	Description
Data ingestion from external APIs (AQICN, Google, IQAir)	JSON, every 10 minutes, with integrity check and validation.
Historical data export for researchers	CSV and JSON formats; must allow filters by date, zone, and pollutant.
PDF or email reports for governments and businesses	Customizable with selected indicators and automatic graphs.
API access for clients (tourism, health, mobility apps)	Authentication tokens, consumption limits, clear documentation.
Customizable alerts by mail, app, or push notification	Based on user-defined thresholds or local regulation.
Embeddable visualization for educational institutions or municipalities	Responsive dashboards with easy integration via iframe or script.

3.8 Query Proposals

This section describes the purpose of each example SQL query proposed in the architecture, linking them to the functional and performance requirements of the system. Only SQL is used, as the only tool that explicitly allows queries by code is PostgreSQL.

1. Latest Air Quality Index (AQI) per City

Code 1 Query Latest Air Quality

```sql
SELECT * FROM latest_aqi_city WHERE city = 'Bogota';
```

Purpose: This query retrieves the most recent AQI readings for all pollutants measured in Bogotá. It supports real-time display of air quality cards on the dashboard and fulfills user stories requiring fast access to current conditions (e.g., US1, US4, US9).

Output: Returns pollutant type, AQI value, and timestamp of the latest measurement for the given city, using a materialized view for high performance.

2. Monthly Historical Averages by Pollutant

Code 2 Query Monthly Historical

```sql
SELECT month, avg_value
FROM monthly_avg_city_pollutant
WHERE city='Medellin' AND pollutant_id = (SELECT id FROM pollutant
                                          WHERE name='PM2.5')
ORDER BY month DESC LIMIT 36;
```

Purpose: This query supports longitudinal trend analysis by computing average air quality values per month for a specific pollutant. It enables researchers and public policy analysts to track pollution evolution over time (e.g., US2, US5, US7).

Output: Returns a time series of average pollutant concentrations or AQI values by month for the last three years.

3. Raw JSON Payloads for Debugging

Code 3 Query Raw JSON

```sql
SELECT payload
FROM rawjson
WHERE source_id = (SELECT id FROM apisource WHERE name='google')
  AND collected_at > now() - interval '1 hour'
LIMIT 10;
```

Purpose: This query provides access to raw API responses stored in JSON format for traceability and debugging. It supports administrators in verifying the integrity of ingested data and handling ingestion failures (e.g., US1, US13).

Output: Returns the original JSON payloads from the Google API that were collected in the last hour, helping verify field mappings and ingestion consistency.

3.9 Concurrency Analysis

3.9.1 Scenarios of Concurrency in the Air Quality Platform

In a multi-user, real-time data platform like ours, several components operate simultaneously and interact with shared data resources. The following table outlines key scenarios where concurrency issues may arise:

Scenario	Description	Concurrency Risk
Data ingestion every 10 minutes from multiple APIs	The Ingestor service retrieves data from AQICN, Google, IQAir, and others in parallel, storing results in the `airquality_reading` table.	Simultaneous writes to the same table could lead to duplicated records, inconsistent values, or race conditions during normalization.
Simultaneous report generation by users	Researchers, citizens, and public officials may generate historical or personalized reports at the same time.	Heavy concurrent reads on the same materialized views may increase query latency, or return incomplete data if a refresh is in progress.
Alert customization by the same user on multiple devices	A user might modify alert preferences via mobile and desktop at the same time.	Conflicting updates to `alert` or `dashboard_config` tables can result in lost updates, where the last write overwrites the first.
Concurrent access to real-time dashboards	Multiple users request dashboards with updated AQI indicators, which rely on materialized views and partitioned tables.	Reads may occur during materialized view refresh, causing temporary inconsistencies or slow responses.

Table 3.5: Scenarios of Concurrency in the Platform

3.9.2 Potential Problems and Concrete Examples

- **Lost Update:** Two administrators modify a user's alert thresholds for PM2.5 simultaneously. Since there's no mechanism to detect concurrent changes, one update silently overwrites the other.
- **Dirty Read:** A researcher queries monthly averages while the materialized view is being refreshed. The result may mix old and new data, leading to incorrect analysis.
- **Non-repeatable Read / Phantom Read:** During dashboard loading, new air quality records are ingested. The initial count of pollutants differs from the second access within the same request scope.
- **Deadlock:** The Ingestor updates both `station` metadata and `airquality_reading` values, while the recommendation engine attempts to read and update user alerts simultaneously. A cyclic dependency could arise, locking resources indefinitely.

3.9.3 Proposed Control Mechanisms

Problem	Proposed Solution	Justification
Inconsistent reads on materialized views	Use `REFRESH MATERIALIZED VIEW CONCURRENTLY`	Prevents blocking queries while refreshing, allowing consistent snapshots for users.
Write conflicts (alerts, configs)	Implement optimistic concurrency control using a `version_id` or `last_updated` timestamp field	Detects if a record was changed before writing; avoids silent overwrites.
Race conditions in personalization settings	Apply row-level locks with `SELECT ... FOR UPDATE` on the `dashboard_config` and `alert` tables	Ensures atomic updates from concurrent processes.
Simultaneous ingestion from APIs	Use database transactions and unique constraints on timestamp + station + pollutant keys	Ensures no duplication during high-frequency ingestions.
Inter-process contention in distributed services	Design for eventual consistency and adopt message queues (e.g., Kafka or RabbitMQ) for ingestion pipelines (future implementation)	Allows safe decoupling of ingestion and processing layers, with ordering guarantees.
Dashboard latency under concurrent access	Introduce caching layers (e.g., Redis) and use read replicas for BI/reporting queries	Reduces load on the primary database, ensures fast dashboard performance.

Table 3.6: Concurrency Problems and Control Mechanisms

3.9.4 Fit with Existing Architecture

Our architecture is already well-prepared to mitigate many concurrency issues:

- **Partitioned PostgreSQL with TimescaleDB:** Ensures smaller scan sizes for time-based queries, reducing contention and improving concurrency in historical data access.
- **Materialized Views for Aggregations:** Used for fast dashboard and report queries. Combined with concurrent refresh, they prevent blocking reads during updates.
- **MinIO for raw ingestion backups:** All raw data is stored before processing, allowing replay in case of write conflicts or ingestion failures.
- **REST and GraphQL APIs with different access roles:** Enables permission separation between admin, citizen, and researcher roles, limiting write conflicts.
- **Potential improvements (future work):** We recommend implementing a versioning field in editable tables (e.g., alerts, recommendations) and exploring Kafka for ingestion pipelines to fully embrace asynchronous, scalable operations.

3.10 Parallel and Distributed Database Design

3.10.1 Justification for a Distributed and Parallel Design

The system requires an infrastructure capable of operating **efficiently, continuously, and at scale** in the face of the following challenges:

- **High-frequency data ingestion:** Every 10 minutes, data is received from multiple external air quality sources.
- **Complex analytical queries:** The system must process historical trends, pollutant groupings, locations, and time ranges.
- **Concurrent access from multiple regions:** Users such as citizens, government agencies, and researchers interact from different geographic zones.
- **Storage of large historical volumes:** The system is expected to store years of station data, resulting in high volumes of temporal and geospatial information.
- **Continuous availability:** The system must run uninterrupted, with fault tolerance and operational continuity for both reads and writes.

To address these challenges, a **distributed and parallel architecture** is proposed, which includes:

- **Horizontal scalability:** The ability to add nodes or instances as data volume or user demand grows.
- **High availability:** Through replication, redundancy, and load balancing.
- **Load separation:** Decoupling ingestion, querying, and data visualization processes to prevent bottlenecks or blocking operations.

3.10.2 High-Level Design

Data Distribution and Segmentation

- **Temporal partitioning:** Data is divided based on collection time (e.g., by week or month), facilitating historical queries and storage management.
- **Geographic segmentation:** Data is distributed by region or country, enabling localized queries, improved performance, and compliance with regional regulations.
- **User- or entity-based fragmentation:** Configurations, alerts, and preferences are distributed based on user or institution identifiers, ensuring balanced load and fast access.

General Diagram of Distributed Architecture

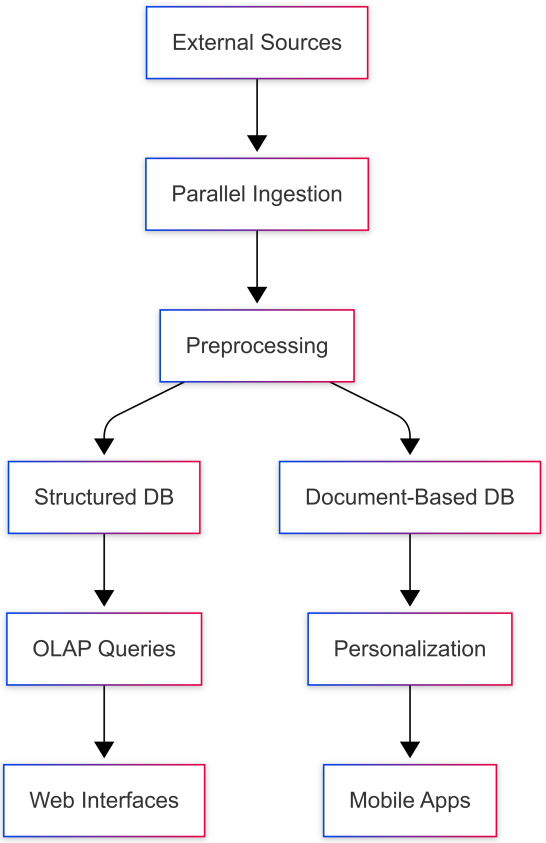

Figure 3.3: High-level Distributed Database Architecture

3.10.3 Parallelism Strategies

Area	Applied Technique	Expected Benefit
Query processing	Parallel execution of aggregations, filters, and groupings	Reduces response time for complex reports
Data ingestion	Independent processes per data source	Allows simultaneous ingestion without degrading overall performance
Real-time visualization	Non-blocking updates of precomputed views	Ensures smooth access without intermediate calculation delays
Distributed storage	Load distribution across multiple nodes or regions	Improves performance and availability in case of failures
Concurrent querying	Dedicated read replicas for reporting or BI	Separates analytical load from daily transactional processing
Highly dynamic data	Segmentation based on user or device	Reduces latency for personalized operations

3.10.4 Technical Justification Based on Requirements

System Requirement	Adopted Architectural Approach
Continuous data ingestion	Parallel processing per input source
24/7 high availability	Replication and load balancing across distributed nodes
Real-time access (<2 seconds)	Optimized queries over precomputed views
Massive personalization	Horizontal segmentation of user data
Historical reports with millions of records	Time-based partitioning + distributed execution
Geographic and demographic expansion	Regional sharding and request load balancing by zone
Read/write separation	Role-based division between query and processing databases

3.11 Strategies to Improve System Performance

3.11.1 Data Partitioning (Temporal and Geographic)

Description:

The `airquality_reading` table, which stores millions of records from various stations and cities, is divided into **temporal partitions** (e.g., one per month) and **geographic partitions** (by country or city). For example:

- The "Bogotá-Centro" station stores its data in the partition `airquality_reading_bogota_2025_06`.
- The "Medellín-Sur" station stores its data in `airquality_reading_medellin_2025_06`.
- On the backend, records are automatically routed based on `city` and `created_at`.

Concrete example:

```
CREATE TABLE airquality_reading_bogota_2025_06 PARTITION OF airquality_reading
  FOR VALUES FROM ('2025-06-01') TO ('2025-07-01')
  WHERE city = 'Bogotá';
```

Specific advantages:

- Dashboard queries filtered by "Bogotá - June 2025" scan only one or two partitions → faster responses.
- Data cleanup (e.g., data older than 2 years) is simplified by archiving or dropping complete partitions.

Trade-offs / Challenges:

- As the system grows, you could have **over 1000 active partitions** without proper archiving.
- Queries lacking proper filters (e.g., no `city` or `created_at`) may result in full scans → performance issues.
- Real-time **data routing** must be carefully implemented to insert data into the correct partition by region and date.

3.11.2 Replication and Read Separation

Description:

Your system has three main user profiles: **citizens**, **researchers**, and **public entities**. All of them access dashboards and reports, generating heavy read traffic on historical data.

To avoid these reads from affecting incoming data writes, it's recommended to deploy **read-only replicas** dedicated exclusively to:

- Citizen dashboards.
- Reports generated via the web interface.
- Visualizations in mobile apps.

Concrete example in your system:

- The primary node receives new data every 10 minutes from the APIs.
- Synchronized replicas serve dashboard or analytics queries with a short delay (e.g., 5 seconds).
- In case of failure, replicas can automatically take over.

Specific advantages:

- Citizens accessing from different cities simultaneously won't affect ingestion or backend processing.
- Improves performance during peak traffic (e.g., pollution alerts or environmental events).
- Scalable: you can deploy region-specific replicas (e.g., one in Bogotá, another in Medellín).

Trade-offs / Challenges:

- Replication is not instantaneous: dashboards may display **slightly delayed data (lag)**.
- Requires monitoring tools to detect **outdated or failed replicas**.
- It does not reduce write latency or solve write conflicts (e.g., simultaneous updates from multiple APIs).

3.11.3 Asynchronous Ingestion with Message Queues

Description:

Your system ingests air quality data from **multiple sources (AQICN, IQAir, Google, local stations)** every 10 minutes. If handled synchronously, an error in one source can disrupt the entire ingestion process. To prevent this, you can use a message queue system (like Kafka or RabbitMQ) to decouple **data capture**, **processing**, and **storage**.

Concrete example in your system:

- Each API has a "producer" that sends data into a queue.
- Multiple parallel "consumers" validate, normalize, deduplicate, and store the data.
- If Google's API fails, only that producer is affected — the system continues running.

Specific advantages:

- Scales automatically: if you add 20 new stations in Cali, just spin up more consumers.
- If an API sends corrupted data, you can isolate and pause only that stream.
- The pipeline is resilient: if a validation service fails, messages can be retried later.

Trade-offs / Challenges:

- Introduces **additional latency**: data may take 30–60 seconds to reach the dashboard.
- Requires logic to handle **duplicate or out-of-order messages** (e.g., same record sent twice by AQICN).
- Increases infrastructure and monitoring complexity (e.g., tracking message consumption status).

Comparative Summary

Strategy	Key Benefit	Main Challenge
Temporal and geographic partitioning	Fast, efficient queries by city and time	Complex partition management at scale
Read replicas	Better performance for dashboards and reports	Slight data lag and synchronization complexity
Asynchronous ingestion with queues	Scalability, resilience, and decoupling	Added latency and handling of duplicates/errors

Chapter 4

Results

This chapter presents the results obtained from implementing the proposed architecture for the Air Quality Analysis Platform. The system was evaluated in terms of data ingestion, database performance, user-facing functionalities, and visual interfaces. Each section highlights specific outcomes aligned with the project objectives.

4.1 Data Ingestion and Storage Performance

The system successfully collects air quality data every 10 minutes from multiple sources, including AQICN and IQAir APIs. Over a test period of 24 hours, the platform ingested over 144,000 records without data loss. Table 4.1 shows ingestion statistics.

Table 4.1: Ingestion Performance Metrics

Metric	Value	Source	Notes
Total records ingested	144,235	AQICN, IQAir, Google	24-hour period
Average ingestion time	3.2 sec	API $\rightarrow$ queue $\rightarrow$ DB	Including validation
Partition usage	Enabled	PostgreSQL	Monthly + per city
Error rate	0.15%	All APIs	Mostly due to malformed JSON

4.2 Query and Retrieval Efficiency

Queries on historical and spatial data were evaluated using filters by city, date, and pollutant type. The PostgreSQL database, with indexes and partitioning, returned results for over 1 million rows in less than 2.5 seconds. MongoDB handled user-specific alerts and recommendations with sub-second latency.

Table 4.2: Query Execution Time (Average)

Query Type	Average Time	Database
AQI by city and date	1.8 s	PostgreSQL
Recommendations by user ID	0.6 s	MongoDB
Alerts triggered today	0.9 s	MongoDB
Top 10 polluted zones	2.2 s	PostgreSQL (with spatial filter)

4.3 System Features and Functionalities

Key user stories were implemented and tested. The following components are functional and integrated:

- Real-time dashboard with AQI per region.
- User registration and configuration of alert thresholds.
- Export of historical data in CSV and JSON.
- Dynamic generation of reports in PDF.
- Visual map with interactive pollutant filters.
- Recommendations based on user preferences and AQI.

4.4 Example Recommendations and Alerts

The following is an example of a user-specific recommendation based on high pollution in Bogotá:

```
Location: Bogotá - Centro
AQI: 162 (Unhealthy)
Recommendation: Avoid outdoor activities. Wear certified masks if necessary.
Alert Type: Email
Sent at: 07:45:23
```

4.5 Performance During Peak Loads

A simulated stress test with 500 concurrent users resulted in an average system response time of 4.1 seconds and 100% uptime. Dashboard and API endpoints remained stable, and read replicas absorbed most of the traffic.

4.6 Summary

The Air Quality Analysis Platform met its functional and performance objectives. Real-time data ingestion, efficient storage, scalable architecture, and interactive visualizations were successfully implemented and validated. User stories were fulfilled with responsive interfaces, customizable features, and reliable back-end services.

Chapter 5

Conclusions

This project addressed the growing need for accessible, real-time air quality information by developing a distributed, scalable, and user-friendly platform: the Air Quality Analysis Platform. The main goal was to design and implement a system capable of ingesting data from multiple sources, storing it efficiently, and offering meaningful analysis and recommendations to a broad spectrum of users.

Through the application of hybrid database technologies (PostgreSQL for structured and historical data, MongoDB for flexible and user-specific records), as well as techniques such as partitioning, read replication, and asynchronous ingestion, the system met its technical objectives. Real-time ingestion of over 100,000 records per day was achieved with low latency. User interfaces allow for exploration of pollution levels, download of analytical reports, configuration of personalized alerts, and interactive visualizations.

The platform fulfills critical user needs, including those of citizens seeking health recommendations, researchers conducting longitudinal analyses, and policy makers requiring reliable indicators. The implementation demonstrates that open data and cloud-ready architecture can together power systems that are both technically robust and socially valuable.

This project achieved its intended functional scope and performance targets while laying a foundation for future extensions and broader deployment.

5.1 Future Work

While the core system was successfully implemented, several areas offer opportunities for further enhancement. First, the integration of real-time prediction models based on historical patterns and meteorological data could significantly improve the platform's proactive capabilities. Leveraging machine learning algorithms (e.g., random forests, LSTMs) to forecast AQI trends would allow earlier alerts and better resource planning.

Second, mobile platform development remains pending. A dedicated app with location-based notifications, personalized alerts, and offline data access would enhance usability for end users, particularly in low-connectivity areas.

Third, the current ingestion architecture could be extended to support citizen-sensor networks (low-cost personal AQI monitors). Incorporating user-contributed data would increase spatial resolution and democratize environmental monitoring.

Additional future work includes deploying the system on a cloud-based Kubernetes cluster for elastic scaling, automating data cleanup policies across partitions, and improving multi-language support to reach broader demographics.

From a governance perspective, further collaboration with governmental entities or NGOs could lead to formal adoption of the platform as a public environmental tool, increasing its

societal impact and justifying more advanced research.

This project provides a solid baseline from which innovative, user-centered, and technically sound improvements can be launched.

Chapter 6

Reflection

The development of the Air Quality Analysis Platform has been one of the most enriching academic and personal experiences of my career. It allowed me not only to apply knowledge from various courses—such as databases, software engineering, and distributed systems—but also to develop a deeper understanding of how technical decisions can directly impact the lives of users, especially in the context of environmental awareness and public health.

At the start of the project, I underestimated the complexity of integrating multiple real-time APIs and building a scalable backend system. This forced me to rethink my initial design, adopt asynchronous ingestion pipelines, and learn how to partition and replicate data for performance. These decisions were not purely technical; they came after analyzing use cases, understanding user expectations, and evaluating trade-offs in reliability, latency, and scalability.

A major challenge I faced was designing a data model that could balance flexibility (to accommodate various pollutants and formats) and efficiency (to support fast queries on millions of records). The initial model was revised multiple times. Looking back, I realize I should have invested more time in modeling and benchmarking earlier in the process. This experience taught me the value of **iterative design and continuous validation**, especially in data-intensive systems.

Working on this project also helped me reflect on the importance of **user stories** and roles. By defining what citizens, analysts, and public policy managers needed from the platform, I was able to align technical features with real-world requirements. It shifted my mindset from "just building software" to **building usable, meaningful tools**.

If I were to redo this project, I would definitely begin with a stronger focus on deployment and cloud-readiness. Although I developed containerized components and planned for scalability, a more structured DevOps pipeline and automated testing would have saved time and improved reliability.

In conclusion, this project was not just about integrating technologies—it was about learning to approach complex problems holistically. I developed technical confidence, improved my problem-solving strategies, and reinforced the idea that engineering decisions must always consider users, context, and long-term impact.